Light

Character Sets (ASCII/Unicode)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. What is a character set?
2. ASCII encoding
3. Unicode and beyond
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Character Sets (1CP2/01 Topic 2: Data)**. This topic covers how computers represent text characters using binary codes. You need to understand 7-bit ASCII encoding, including how to convert between characters and their binary/denary codes, and why character sets like Unicode exist for representing a wider range of characters.

Exam questions typically ask you to **convert** characters to ASCII codes (and vice versa), **explain** how ASCII represents characters, and **compare** ASCII with Unicode. Questions are typically worth 2—4 marks and may involve calculations using the ASCII table provided.

Why character sets matter

Computers can only store and process binary data — sequences of 0s and 1s. To represent text (letters, digits, symbols), we need a system that assigns a unique binary code to each character. This is what a **character set** provides.

- **Standardisation:** Character sets ensure that text created on one computer can be read correctly on another
- **Communication:** When you send a message, the binary codes are transmitted and decoded back into characters

- **Storage:** Text files store the binary codes, not the visual shapes of characters
- **Programming:** Understanding ASCII helps you work with strings and character manipulation in code

Without agreed character sets, the letter 'A' on your computer might display as something completely different on someone else's computer.

What you must know

This checklist covers the key exam-visible concepts from the Edexcel specification (2.2.1).

- **Define** what a character set is and why computers need them
- **Explain** how 7-bit ASCII encodes characters using binary patterns
- **State** that 7-bit ASCII can represent 128 different characters ($2^7 = 128$)
- **Convert** characters to their ASCII codes (denary and binary) using a reference table
- **Convert** ASCII codes back to their corresponding characters
- **Calculate** the binary representation of a character given its ASCII code
- **Explain** why Unicode was developed (to represent more characters than ASCII allows)

> **Specification link:** This topic covers Edexcel 1CP2 specification section 2.2.1 — "understand how computers encode characters using 7-bit ASCII."

Toolkit

These are the essential reference tables you need for exam questions. In the exam, you will be given an ASCII table, but understanding the patterns helps you work faster and check your answers.

Key ASCII codes to remember

While you will have a table in the exam, knowing these key codes saves time and helps you spot errors in your working:

Character	Denary	Binary (7-bit)	Notes
Space	32	0100000	First printable character
0	48	0110000	Digits 0—9 are codes 48—57
9	57	0111001	Last digit
A	65	1000001	Uppercase A—Z are codes 65—90
Z	90	1011010	Last uppercase letter
a	97	1100001	Lowercase a—z are codes 97—122
z	122	1111010	Last lowercase letter

ASCII pattern shortcuts

ASCII was cleverly designed with patterns that make certain operations easy. These shortcuts are useful for checking your answers and for programming tasks:

Uppercase to lowercase

To convert uppercase to lowercase:
Add 32 to the ASCII code
Example: 'A' (65) → 'a' (97)
65 + 32 = 97 ✓

This works because the binary difference is just bit 5 (the 32s place).

Digit character to value

To get the numeric value from a digit character:
Subtract 48 from the ASCII code
Example: '7' (55) → 7
55 − 48 = 7 ✓

The character '0' has code 48, so subtracting 48 gives the actual number.

Finding letter positions

To find the code for any letter, use these anchor points:

Uppercase: 'A' = 65, so any uppercase letter = 65 + (position in alphabet − 1)
Example: 'E' is the 5th letter, so 'E' = 65 + 4 = 69

Lowercase: 'a' = 97, so any lowercase letter = 97 + (position in alphabet − 1)
Example: 'e' is the 5th letter, so 'e' = 97 + 4 = 101

Powers of 2 for character sets

Bits	Total characters	Character set
7	$2^7 = 128$	7-bit ASCII
8	$2^8 = 256$	Extended ASCII
16	$2^{16} = 65{,}536$	Early Unicode
21	$2^{21} = 2{,}097{,}152$	Unicode (current)

1. What is a character set?

A **character set** is a collection of characters that a computer can recognise and use, where each character is assigned a unique binary code. The character set defines which characters are available and what binary pattern represents each one.

1.1 Why do we need character sets?

Computers work only with binary — they cannot directly understand letters, symbols, or even digits as we see them. A character set provides a mapping between human-readable characters and the binary codes that computers store and process.

Think of a character set like a codebook or dictionary that both the sender and receiver must agree on. When you type the letter 'A', your computer looks up 'A' in the character

set, finds its code (65 in ASCII), and stores that number. When displaying the text, the computer reads the number 65, looks it up in the character set, and draws the letter 'A' on screen.

A character set includes:

- Letters (uppercase and lowercase)
- Digits (0—9)
- Punctuation marks (. , ! ? etc.)
- Special symbols (@ # $ % etc.)
- Control characters (newline, tab, etc.)
- Space character

Benefits of standardised character sets:

- **Interoperability:** Files can be shared between different computers and operating systems
- **Communication:** Text transmitted over networks is decoded correctly by the receiver
- **Consistency:** Programs can rely on predictable character codes
- **Sorting:** Characters can be sorted in a defined order based on their codes

1.2 How character encoding works

When you type a character on a keyboard, the computer stores its corresponding binary code in memory. When displaying text, the computer reads the binary codes and renders the appropriate character shapes on screen. This process happens billions of times every second across all the computers in the world.

Every key press on your keyboard sends a signal that gets converted to an ASCII code. Every character you see on screen started as a binary number that was looked up in a character set to determine what shape to draw. Understanding this process helps you appreciate how fundamental character sets are to computing.

Character encoding example:

`H` `i` → `72` `105` → `1001000` `1101001`

The word "Hi" becomes two ASCII codes (72, 105), stored as binary patterns.

Worked Example 1 – Explaining character sets

Question: Explain what a character set is and why it is needed. [2 marks]

Answer:

A character set is a collection of characters where each character has a unique binary code assigned to it. **(1)**

It is needed because computers can only process binary data, so text characters must be converted to binary codes for storage and processing. **(1)**

Exam tip: Always mention "unique binary code" — this is the key point that earns marks.

Worked Example 2 – Calculating characters from bits

Question: A character set uses 7 bits to represent each character. Calculate how many different characters can be represented. [1 mark]

Formula: Total characters = 2^n where n = number of bits
Calculation: $2^7 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2$ = **128 characters (1)**

Exam tip: Remember: $2^7 = 128$. This is the number of characters in standard ASCII.

2. ASCII encoding

ASCII (American Standard Code for Information Interchange) is a character encoding standard that uses 7 bits to represent each character. It was developed in the 1960s and became the foundation for text representation in computers.

2.1 Structure of ASCII

With 7 bits, ASCII can represent $2^7 = 128$ different characters, numbered 0 to 127 in denary. These include:

Control characters (0–31)

Non-printable characters used for controlling devices and formatting:

- 0: Null (NUL)
- 9: Tab
- 10: Line feed (newline)
- 13: Carriage return
- 27: Escape

Printable characters (32–126)

Characters that display on screen:

- 32: Space
- 48—57: Digits 0—9
- 65—90: Uppercase A—Z
- 97—122: Lowercase a—z
- Various punctuation and symbols

Delete (127)

The final ASCII code:

- 127: Delete (DEL)
- Originally used to mark deleted characters on paper tape

2.2 History and importance of ASCII

ASCII was developed in the early 1960s by the American Standards Association (now ANSI). It became the dominant character encoding for early computers and the internet, and remains important today because:

- **Foundation of modern encoding:** Unicode and other character sets are built upon ASCII's original 128 characters
- **Network protocols:** Many internet protocols like HTTP and SMTP were designed around ASCII text
- **Programming languages:** Most programming languages use ASCII for source code, variable names, and keywords
- **Cross-platform compatibility:** ASCII text files can be read on virtually any computer system

The design of ASCII was carefully planned to make certain operations easier. For example, letters are arranged consecutively (A=65, B=66, C=67...) so they can be sorted alphabetically by comparing their codes.

2.3 ASCII reference table (printable characters)

Dec	Char	Dec	Char	Dec	Char	Dec	Char	Dec	Char
32	(space)	48	0	64	@	80	P	96	`
33	!	49	1	65	A	81	Q	97	a
34	"	50	2	66	B	82	R	98	b
35	#	51	3	67	C	83	S	99	c
36	$	52	4	68	D	84	T	100	d
37	%	53	5	69	E	85	U	101	e
38	&	54	6	70	F	86	V	102	f
39	'	55	7	71	G	87	W	103	g
40	(	56	8	72	H	88	X	104	h
41	)	57	9	73	I	89	Y	105	i
42	*	58	:	74	J	90	Z	106	j
43	+	59	;	75	K	91	[	107	k
44	,	60	<	76	L	92	\	108	l
45	-	61	=	77	M	93	]	109	m
46	.	62	>	78	N	94	^	110	n
47	/	63	?	79	O	95	_	111	o
								112	p
								113—122	q—z

2.4 Converting between characters and ASCII codes

In the exam, you may be asked to convert a character to its ASCII code (denary or binary), or convert an ASCII code back to its character. You will be given an ASCII table to use.

Character → Denary → Binary

1. Find the character in the ASCII table
2. Read its denary (decimal) code
3. Convert the denary to 7-bit binary

> **Example: Convert 'K' to binary**
> Step 1: Find 'K' in table → denary 75
> Step 2: Convert 75 to binary:
> 75 = 64 + 8 + 2 + 1 = 1001011
> Answer: **1001011**

Binary → Denary → Character

1. Convert the binary to denary
2. Find the denary code in the ASCII table
3. Read the corresponding character

> **Example: Convert 1010111 to character**
> Step 1: Convert to denary:
> 1010111 = 64 + 16 + 4 + 2 + 1 = 87
> Step 2: Find 87 in table → 'W'
> Answer: **W**

2.5 Practical applications of ASCII knowledge

Understanding ASCII codes has practical applications in computing beyond just exam questions:

In programming:

- **String comparison:** When sorting text alphabetically, programs compare ASCII codes
- **Input validation:** Checking if a character is a letter, digit, or symbol
- **Case conversion:** Converting between uppercase and lowercase by adding/subtracting 32
- **Password strength:** Checking for a mix of character types

In data handling:

- **Text file storage:** Understanding how text is stored in memory and on disk
- **Data transmission:** How characters are sent over networks
- **Encryption:** Many simple ciphers work by manipulating ASCII codes
- **Debugging:** Reading hex dumps of files to identify characters

2.7 The uppercase/lowercase relationship

One useful pattern in ASCII is that uppercase and lowercase letters are exactly 32 positions apart. This is because the only difference in their binary codes is bit 5 (the 32s

column).

Character	Denary	Binary (7-bit)
A	65	1000001
a	97	1100001

The difference: 97 − 65 = 32 (bit 5 changes from 0 to 1)

Exam tip: To convert uppercase to lowercase, add 32. To convert lowercase to uppercase, subtract 32. This works for all letters A—Z and a—z.

Worked Example 3 — Character to ASCII code

Question: Using the ASCII table, state the denary code for the character 'M'. [1 mark]

Method: Look up 'M' in the ASCII table.
Answer: 77 **(1)**

Exam tip: 'A' = 65, so 'M' = 65 + 12 = 77 (M is the 13th letter, but we start counting from 0).

Worked Example 4 — ASCII code to binary

Question: The ASCII code for a character is 83. Convert this to a 7-bit binary number. Show your working. [2 marks]

Step 1: Break down 83 using powers of 2:
83 = 64 + 16 + 2 + 1 **(1)**
Step 2: Write in binary (place values: 64, 32, 16, 8, 4, 2, 1):
= **1010011 (1)**

Exam tip: Check: 64 + 16 + 2 + 1 = 83 ✓. The character with code 83 is 'S'.

Worked Example 5 — Binary to character

Question: The binary code 1000100 represents a character in ASCII. State which character this represents. Show your working. [2 marks]

Step 1: Convert binary to denary:

1000100 = 64 + 4 = 68 **(1)**
Step 2: Look up 68 in ASCII table:
68 = **'D' (1)**

Exam tip: 'A' = 65, so 68 = 65 + 3, which is the 4th letter: D.

Worked Example 6 — Encoding a word

Question: Using the ASCII table, write the denary codes for the word "Cat". [2 marks]

Step 1: Look up each character:
C = 67
a = 97
t = 116 **(1)**
Answer: 67, 97, 116 **(1)**

Exam tip: Note that 'C' (uppercase) has a different code from 'c' (lowercase). 'C' = 67, 'c' = 99.

Worked Example 7 — Case conversion calculation

Question: The ASCII code for 'G' is 71. Calculate the ASCII code for 'g'. Explain your method. [2 marks]

Method: Lowercase letters have ASCII codes 32 higher than their uppercase equivalents. **(1)**
Calculation: 71 + 32 = **103 (1)**

Exam tip: This relationship (difference of 32) applies to all letters from A—Z and a—z.

Worked Example 8 — File size calculation

Question: A text file contains 500 characters. Each character is stored using 7-bit ASCII. Calculate the minimum file size in bits and bytes. [2 marks]

Calculation:
Size in bits = 500 × 7 = **3,500 bits (1)**

Size in bytes = 3,500 ÷ 8 = **437.5 bytes** (round up to 438 bytes) **(1)**

Exam tip: In practice, ASCII is often stored as 8 bits per character (with an unused bit), so 500 characters = 500 bytes. Read the question carefully to see if it specifies 7-bit or 8-bit storage.

3. Unicode and beyond

While ASCII was groundbreaking, it has a significant limitation: with only 128 characters, it cannot represent the characters used in many of the world's languages. **Unicode** was developed to solve this problem and has become the standard for modern text encoding.

3.1 Limitations of ASCII

ASCII was designed primarily for English text in the United States during the 1960s. At that time, computer memory was expensive and international communication was limited, so a small character set was practical. It includes only:

- 26 uppercase letters (A—Z)
- 26 lowercase letters (a—z)
- 10 digits (0—9)
- Punctuation and common symbols
- Control characters

ASCII cannot represent:

- Accented characters (é, ñ, ü, ç)
- Greek letters (α, β, γ, Ω)
- Cyrillic characters (Д, Ж, Щ)
- Arabic script (ع, ض, ص)
- Chinese characters (中, 文, 字)
- Japanese characters (日, 本, 語)
- Mathematical symbols (Σ, ∞, ≠)
- Emojis (😀, 🎉, 🌍)

This limitation became increasingly problematic as computers spread globally and the internet connected people across different countries and languages. Documents, emails, and websites needed to display text in many different languages simultaneously.

3.2 Unicode as a solution

Unicode is a character encoding standard designed to represent virtually every character from every writing system in the world. It assigns a unique code to over 140,000 characters, and the standard continues to grow with each new version. Unicode was first published in 1991 by the Unicode Consortium, an organisation dedicated to standardising text representation. Today, Unicode is used by virtually all modern software, websites, and operating systems.

ASCII

- 7 bits per character
- 128 characters maximum

- English characters only
- Developed in 1963
- Smaller file sizes
- Limited international support

Unicode

- Variable bits (8, 16, or 32)
- Over 140,000 characters
- All world languages
- Developed in 1991
- Larger file sizes
- Full international support

Key point: Unicode is backwards compatible with ASCII. The first 128 Unicode characters (0—127) are identical to ASCII, so ASCII text files can be read as Unicode without conversion. This was a deliberate design decision to ensure existing ASCII documents remained usable.

3.3 Why file sizes differ

Because Unicode uses more bits per character than ASCII, text files encoded in Unicode are generally larger than the same text encoded in ASCII. This is an important consideration when storing or transmitting large amounts of text data.

Example: Storing the word "Hello" (5 characters)

Encoding	Bits per character	Total size
7-bit ASCII	7	5 × 7 = 35 bits
Unicode (16-bit)	16	5 × 16 = 80 bits

The Unicode version is more than twice the size!

3.4 When to use ASCII vs Unicode

In modern computing, Unicode has largely replaced ASCII for most purposes. However, understanding both is important:

ASCII is still used for:

- Simple configuration files
- Programming source code
- Network protocols (HTTP headers)
- Legacy systems
- Embedded systems with limited memory

Unicode should be used for:

- Documents with international text
- Web pages and applications
- User-facing content
- Databases storing multilingual data
- Modern software development

For the GCSE exam, you need to understand the principles of both ASCII and Unicode, but the focus is on 7-bit ASCII. Questions about Unicode typically ask you to explain why it was developed or to compare file sizes between ASCII and Unicode encoding.

Worked Example 9 — ASCII limitations

Question: Explain one limitation of ASCII. [2 marks]

Answer:
ASCII can only represent 128 different characters **(1)**, which is not enough to include characters from other languages such as Chinese, Arabic, or accented European characters. **(1)**

Exam tip: Always give a specific example of what ASCII cannot represent (e.g., "Chinese characters" rather than just "other characters").

Worked Example 10 — Why Unicode exists

Question: Explain why Unicode was developed. [2 marks]

Answer:
ASCII only supports 128 characters, which limits it to basic English text. **(1)**
Unicode was developed to represent characters from all world languages, including Chinese, Arabic, and special symbols, enabling international communication. **(1)**

Exam tip: Mention both the problem (ASCII's limitation) and the solution (Unicode's capability).

Worked Example 11 — Comparing file sizes

Question: A document contains 1,000 characters. Calculate the file size difference between storing it in 7-bit ASCII versus 16-bit Unicode. Give your answer in bytes. [3 marks]

ASCII size: 1,000 × 7 = 7,000 bits = 875 bytes **(1)**
Unicode size: 1,000 × 16 = 16,000 bits = 2,000 bytes **(1)**

Difference: 2,000 − 875 = **1,125 bytes (1)**

Exam tip: Show your working clearly. Convert bits to bytes by dividing by 8.

Worked Example 12 — ASCII and Unicode compatibility

Question: A programmer saves a file containing only English letters and digits. Explain why the file would work correctly whether opened as ASCII or Unicode. [2 marks]

Answer:
Unicode is backwards compatible with ASCII. **(1)**
The first 128 Unicode character codes (0—127) are identical to ASCII codes, so English letters and digits have the same codes in both systems. **(1)**

Exam tip: "Backwards compatible" is a key term — it means Unicode includes all of ASCII as a subset.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by section and difficulty level.

#	Topic	Question type	Marks	Key skill
1	Character sets	Explain definition and purpose	2	Define character set + state need
2	Bits to characters	Calculate 2^n	1	Apply $2^7 = 128$
3	Character to code	Look up ASCII table	1	Table reading
4	Code to binary	Convert denary to 7-bit binary	2	Binary conversion
5	Binary to character	Convert and look up	2	Binary → denary → character
6	Encoding a word	Multiple character lookup	2	Sequential table reading
7	Case conversion	Calculate with +32 rule	2	Apply uppercase/lowercase pattern

#	Topic	Question type	Marks	Key skill
8	File size	Calculate bits and bytes	2	Characters × bits ÷ 8
9	ASCII limitations	Explain with example	2	128 character limit + examples
10	Why Unicode	Explain development reason	2	Problem + solution structure
11	Size comparison	Calculate ASCII vs Unicode	3	Compare file sizes
12	Compatibility	Explain backwards compatibility	2	First 128 codes identical

Quick reference

Key terms and definitions

Term	Definition (mark-scheme style)
Character set	A collection of characters where each has a unique binary code assigned to it
ASCII	American Standard Code for Information Interchange; a 7-bit character encoding standard
7-bit ASCII	Uses 7 bits per character, allowing 128 different characters (0—127)
Unicode	A character encoding standard that can represent characters from all world languages
Character encoding	The process of converting characters into binary codes for computer storage
Control character	A non-printable ASCII character used for formatting or device control (e.g., newline, tab)
Printable character	An ASCII character that produces a visible symbol when displayed (codes 32—126)
Backwards compatible	When a newer system can work with data from an older system; Unicode includes ASCII as a subset

Essential ASCII values

Range	Characters	Key codes to know
0—31	Control characters	10 = newline, 13 = carriage return
32	Space	First printable character
48—57	Digits 0—9	'0' = 48, '9' = 57
65—90	Uppercase A—Z	'A' = 65, 'Z' = 90
97—122	Lowercase a—z	'a' = 97, 'z' = 122
127	Delete (DEL)	Last ASCII code

Key formulas

Formula	Purpose
2^n characters	Calculate how many characters n bits can represent
Uppercase + 32 = Lowercase	Convert uppercase letter to lowercase
Lowercase − 32 = Uppercase	Convert lowercase letter to uppercase
Digit character − 48 = numeric value	Get number from digit character (e.g., '5' → 5)
Characters × bits per char ÷ 8	Calculate file size in bytes

Typical pitfalls

Pitfall 1 — Confusing 7-bit and 8-bit.
Wrong: "ASCII uses 8 bits per character."
Right: Standard ASCII uses 7 bits (128 characters). Extended ASCII uses 8 bits (256 characters). The exam focuses on 7-bit ASCII.

Pitfall 2 — Forgetting case sensitivity.
Wrong: "'A' and 'a' have the same ASCII code."
Right: 'A' = 65 and 'a' = 97. They are different characters with different codes (difference of 32).

Pitfall 3 — Wrong character range.
Wrong: "ASCII can represent 127 characters."
Right: ASCII can represent 128 characters (codes 0—127). Remember $2^7 = 128$, not 127.

Pitfall 4 — Digit confusion.
Wrong: "The ASCII code for 5 is 5."
Right: The ASCII code for the character '5' is 53. The digit characters start at code 48 ('0'), not 0.

Pitfall 5 — Binary conversion errors.
Wrong: Writing 65 as 1000001 (7 bits correct) but forgetting to show working.
Right: Always show: 65 = 64 + 1 = 1000001. Working earns marks even if the final answer has an error.

Pitfall 6 — Vague limitation answers.
Wrong: "ASCII doesn't have enough characters."
Right: "ASCII only has 128 characters, which cannot include Chinese characters, Arabic script, or accented letters like é."

Pitfall 7 — Confusing Unicode details.
Wrong: Trying to explain UTF-8 or UTF-16 encoding in detail.
Right: At GCSE, you only need to know that Unicode exists to represent more characters than ASCII. Don't go into encoding details.

Pitfall 8 — File size calculation errors.
Wrong: Forgetting to divide by 8 when converting bits to bytes.
Right: Bits ÷ 8 = Bytes. For 7-bit ASCII: characters × 7 ÷ 8 = size in bytes.

Pitfall 9 — Ignoring control characters.
Wrong: "All 128 ASCII characters are letters and symbols."
Right: Codes 0—31 are control characters (non-printable). Only codes 32—126 are printable characters.

Pitfall 10 — Forgetting space is a character.
Wrong: Not counting spaces when calculating file size.
Right: Space is ASCII code 32 and takes 7 bits like any other character. "Hello World" is 11 characters (including the space).

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.